



Interview Preparation Guide

A complete backend engineering study guide — organized from handwritten notes. Covers Node.js, Express, JWT, MongoDB, MySQL, Redis, BullMQ, Nginx, AWS, System Design, and Next.js.

By Suraj · 2026 Edition

Table of Contents

- Ch 1. How to Answer Interview Questions** — The ADERT Framework
- Ch 2. Backend Fundamentals** — REST, HTTP, Node.js architecture
- Ch 3. CommonJS & JavaScript Modules** — require, exports, scope, var/let/const
- Ch 4. Express.js Deep Dive** — Routing, Middleware, Error handling
- Ch 5. JWT & Authentication** — Tokens, Signing, RBAC, Security
- Ch 6. MongoDB** — Documents, CRUD, Aggregation, Indexes
- Ch 7. MySQL & Relational Databases** — ACID, Joins, Indexing, Transactions
- Ch 8. Redis & Caching** — Strategies, TTL, Session storage
- Ch 9. BullMQ & Job Queues** — Worker queues, Producers, Consumers
- Ch 10. Nginx & Reverse Proxy** — Deployment, Load balancing, SSL
- Ch 11. AWS Deployment** — EC2, IAM, S3, VPC, RDS
- Ch 12. System Design** — DNS, CDN, Scalability, DB design
- Ch 13. Next.js Interview Prep** — App Router, SSR, SSG, API Routes

The ADERT Framework

Before diving into technical topics, the most important skill is **how you communicate your knowledge**. Your notes describe a 5-step answer framework. Here it is, formalised.

The ADERT Framework

Every strong interview answer has these parts, in order:

A — Acknowledge + Align

Briefly show you understood the question. This builds rapport and buys you a second to think.

Example: *"Yeah good question — this usually confuses people because Node.js is single-threaded."*

D — Direct Answer (1 line)

Give the core truth in one clean sentence. No waffle. Example: *"Node.js is single-threaded for JS execution but handles concurrency via the event loop and non-blocking I/O."*

E — Explain with Mental Model

Show HOW you think. This is what separates seniors from juniors. Draw an analogy, explain the mechanism. Example: *"Instead of creating a new thread per request like Apache does, Node delegates I/O to the OS/thread pool and keeps the main thread free."*

R — Real-world scenario

Prove you've used this, not just read about it. Example: *"In Dawa Saathi I use BullMQ so the HTTP response returns instantly while the Claude API call runs in a worker."*

T — Tradeoff / Limitation

Show senior thinking. Every technology has tradeoffs. Name them. Example: *"The limitation is CPU-heavy tasks block the event loop — in that case we use worker threads or external services."*

■ **Interview Tip:** After your answer, **pause**. Don't keep talking. Give the interviewer space to follow up. Over-explaining signals nervousness.

Practice Trick

For any concept, force yourself to say: *"If I had to explain this to a junior teammate..."* That exact tone — calm, senior, teacherly — is the perfect interview tone. Not "I read that..." or "I think it's..." but *"So what's happening here is..."*

Common Mindset Mistakes

- ■ Trying to give the 'correct' answer → ■ Give a *thoughtful* answer
- ■ Dumping everything you know → ■ Be selective, let them ask follow-ups
- ■ Skipping tradeoffs → ■ Always name one limitation
- ■ Being vague on real experience → ■ Name your actual project (Dawa Saathi, Piiloopu)

What is REST?

REST (Representational State Transfer) is an architectural style for designing networked APIs. Your notes correctly identify it as *"Constraint-based Stateless Transfer"*. It's not a protocol — it's a set of constraints that, when followed, produce predictable, scalable APIs.

Client–Server Separation

Frontend and backend are completely decoupled. They communicate only through a well-defined interface. This lets you swap the frontend without touching the API.

Statelessness

Every request from the client must contain ALL the information needed to process it. The server stores zero session state between requests. This is why JWTs are sent on every request.

Resource-based URLs

URLs identify nouns (resources), not verbs (actions). *GET /users/42* is REST. *GET /getUser?id=42* is not.

Uniform Interface

Consistent HTTP methods across all resources: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).

Cacheability

Responses should declare whether they can be cached. This reduces load and improves speed.

Layered System

The client doesn't need to know if it's talking to the real server or a proxy/load balancer. Nginx sits in front without the client caring.

■ **Interview Tip:** Interviewers often ask: "What makes an API truly RESTful?" The answer they want is *STATELESSNESS + UNIFORM INTERFACE*. Most 'REST APIs' in production are actually just *JSON-over-HTTP*, not truly RESTful.

HTTP Methods Cheat Sheet

Method	Action	Idempotent?	Body?
GET	Read resource	Yes	No
POST	Create resource	No	Yes

PUT	Replace resource	Yes	Yes
PATCH	Partial update	No	Yes
DELETE	Delete resource	Yes	No

HTTP Status Codes — The Important Ones

- **200 OK** — Standard success
- **201 Created** — Resource successfully created (POST)
- **204 No Content** — Success, but no body (DELETE)
- **400 Bad Request** — Client sent invalid data
- **401 Unauthorized** — Not authenticated — send credentials
- **403 Forbidden** — Authenticated but not allowed
- **404 Not Found** — Resource doesn't exist
- **409 Conflict** — Duplicate entry, version mismatch
- **422 Unprocessable Entity** — Validation error
- **429 Too Many Requests** — Rate limited
- **500 Internal Server Error** — Something broke on the server
- **503 Service Unavailable** — Server is down/overloaded

Node.js Architecture — Why It's Fast

Node.js is **single-threaded for JavaScript execution**, but it handles thousands of concurrent connections using a **non-blocking I/O model** powered by the **libuv event loop**.

How it works:

- Request arrives → pushed onto the Call Stack
- If the operation is I/O (DB query, file read, HTTP call) → Node hands it to libuv and moves on
- libuv uses OS-level async I/O or a thread pool (for things like file I/O)
- When I/O completes → callback is pushed to the Event Queue
- Event Loop checks: is Call Stack empty? Yes → execute next callback

■ **Interview Tip:** "If Node is single-threaded, how does it handle 10,000 concurrent requests?" — Answer: It doesn't **BLOCK**. While waiting for DB/network I/O, the event loop is free to accept new requests. The bottleneck is **CPU-heavy work**, not I/O.

Event Loop Phases (simplified):

- Phase 1: Timers (setTimeout, setInterval)
- Phase 2: I/O callbacks
- Phase 3: Idle/prepare
- Phase 4: Poll (wait for I/O)
- Phase 5: Check (setImmediate)
- Phase 6: Close callbacks

■ **Interview Tip:** Microtasks (*Promise.then*, *queueMicrotask*) run *BETWEEN* each phase, not inside them. This is why *Promise* callbacks execute before *setTimeout* callbacks.

require, exports, scope, var/let/const

Why Modules Exist

Without modules, every JS file shares the same global scope. Variables clash, code becomes spaghetti. Modules solve this by giving each file its own scope, and letting files explicitly export/import what they need.

CommonJS (CJS) — The Node.js Default

Your notes show you understand CJS well. Here's the clean version:

```
// math.js — exporting
function add(a, b) { return a + b; }
module.exports = { add };           // export as object
// OR: module.exports = add;        // export just the function

// app.js — importing
const { add } = require('./math');  // destructure
const math = require('./math');     // whole object
console.log(add(2, 3));              // 5
```

How require() works internally:

- Node wraps each file in an IIFE: (function(exports, require, module, __filename, __dirname) { ... })
- This gives every file its own scope — your top-level variables are NOT global
- require() caches modules — calling require('./math') twice returns the same object
- module.exports is an empty object by default

■ **Interview Tip:** Interviewers love: "What happens when you require the same module twice?" → Node caches it. The module code runs ONCE. The same object reference is returned both times.

ES Modules (ESM) — The Modern Way

```
// math.mjs (or package.json: "type": "module")
export function add(a, b) { return a + b; }
export default 42;

// app.mjs
import { add } from './math.mjs';  // named import
import num from './math.mjs';      // default import
```

Feature	CommonJS	ES Modules
Syntax	require / module.exports	import / export
Loading	Synchronous (blocks)	Asynchronous

Top-level await	No	Yes
Tree-shaking	No	Yes (bundlers)
Default in Node	Yes (older)	With "type":"module"

var vs let vs const

Your notes captured this. Here's the definitive comparison:

	var	let	const
Scope	Function	Block	Block
Hoisted?	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Re-declare?	Yes	No	No
Re-assign?	Yes	Yes	No
Use in 2026?	■ Avoid	■ Yes	■ Prefer

■ **Interview Tip:** TDZ = Temporal Dead Zone. *let* and *const* ARE hoisted but you can't access them before declaration. That's why you get a *ReferenceError*, not *undefined* like *var*.

Routing, Middleware, Error Handling

What is Express?

Express is a **minimal, unopinionated web framework built on top of Node's http module**. It adds routing, middleware, and request/response abstractions so you can build REST APIs without manually parsing URLs, headers, or bodies. Your notes describe it accurately as a 'thin layer on top of Node's http.'

Minimal Express Server

```
const express = require('express');
const app = express();

app.use(express.json());           // parse JSON bodies

app.get('/users/:id', (req, res) => {
  const { id } = req.params;       // URL param
  const { page } = req.query;      // ?page=2
  res.status(200).json({ id, page });
});

app.listen(3000, () => console.log('Running on 3000'));
```

Middleware — The Core Concept

Middleware is a function with signature **(req, res, next)**. Express is basically a chain of middleware functions that execute top-to-bottom. Each one can: modify req/res, end the request, or call next() to pass to the next.

```
// Logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();           // ← MUST call this or request hangs forever
});

// Auth middleware (stops chain if unauthorized)
const requireAuth = (req, res, next) => {
  const token = req.headers.authorization;
  if (!token) return res.status(401).json({ error: 'No token' });
  next();
};

app.get('/protected', requireAuth, (req, res) => {
  res.json({ data: 'secret' });
});
```

■ **Interview Tip:** "What happens if you don't call `next()` and don't send a response?" → The request hangs. The client waits until timeout. This is a common bug.

Error Handling Middleware

Express has a special 4-argument error middleware. It MUST have exactly `(err, req, res, next)`:

```
// Regular routes above...

// Error handler – MUST be registered LAST
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    error: err.message || 'Internal server error'
  });
});

// To trigger it from any route:
app.get('/boom', (req, res, next) => {
  next(new Error('Something broke')); // passes to error handler
});
```

Route Organization with Router

```
// routes/users.js
const router = require('express').Router();

router.get('/', getUsers);
router.get('/:id', getUserById);
router.post('/', createUser);
router.patch('/:id', updateUser);
router.delete('/:id', deleteUser);

module.exports = router;

// app.js
app.use('/users', require('./routes/users'));
```

■ **Interview Tip:** Common interview: "Difference between `app.use()` and `app.get()`?" → `app.use()` matches ANY HTTP method and partial paths. `app.get()` matches only GET on exact path.

Tokens, Signing, RBAC, Security

JWT Structure

A JWT (JSON Web Token) is **three Base64URL-encoded parts separated by dots: Header.Payload.Signature**. Your notes correctly state it is **signed, NOT encrypted by default**. Anyone can decode the payload — but they cannot forge it without the secret key.

```
// Header (algorithm + token type)
{ "alg": "HS256", "typ": "JWT" }

// Payload (claims – userId, role, expiry)
{ "sub": "user_42", "role": "admin", "exp": 1735689600 }

// Signature
HMACSHA256(base64(header) + "." + base64(payload), SECRET_KEY)
```

Standard Claims (Registered):

- sub — subject (user ID)
- exp — expiration (Unix timestamp)
- iat — issued at
- iss — issuer
- aud — audience (who it's for)

HS256 vs RS256

	HS256 (Symmetric)	RS256 (Asymmetric)
Keys	1 secret key	Private key (sign) + Public key (verify)
Use case	Single service	Microservices, 3rd party verification
If key leaks	Everything compromised	Only signing compromised

JWT Security — What Your Notes Covered

Your notes list three guarantees JWT provides:

- **Integrity** — If anyone modifies the header or payload, the signature verification fails. Tampered = rejected.
- **Authentication** — Only the holder of the secret key could have generated a valid signature.
- **Stateless** — Server doesn't need a DB lookup to validate — just re-compute the signature.

What JWT does NOT do:

- Encryption — payload is readable by anyone (don't put passwords in it)
- Revocation — once issued, you can't invalidate a JWT until it expires (use short expiry + refresh token)

Access Token + Refresh Token Pattern

```
// Access token: short-lived (15 min), sent on every request
// Refresh token: long-lived (7 days), stored in httpOnly cookie

// Login
POST /auth/login
→ { accessToken: "...", refreshToken: "..." }

// Every API request
GET /api/data
Authorization: Bearer

// When access token expires:
POST /auth/refresh
Cookie: refreshToken=...
→ { accessToken: "new token" }

// Logout: delete refresh token from DB/Redis
```

RBAC — Role-Based Access Control

Your notes describe this accurately. Instead of assigning permissions to individual users, you assign permissions to **roles**, and assign roles to users. This scales — changing a role's permissions automatically affects all users with that role.

```
// JWT payload with role
{ "sub": "user_42", "role": "admin" }

// Middleware to check role
const requireRole = (role) => (req, res, next) => {
  if (req.user.role !== role)
    return res.status(403).json({ error: 'Forbidden' });
  next();
};

app.delete('/admin/users/:id', requireAuth, requireRole('admin'), deleteUser);
```

■ **Interview Tip:** "Why use RBAC instead of checking `userId` in every route?" → It scales. With RBAC you change one role definition; with user-level permissions you'd update every user.

What is MongoDB?

MongoDB is a **document-oriented NoSQL database**. Instead of rows in tables, it stores **BSON documents** (Binary JSON) in **collections**. Your notes correctly identify: schema-flexible, good for high-traffic scalable apps, and that joins are supported via **\$lookup** but are limited compared to SQL.

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
JOIN	\$lookup (aggregation)
INDEX	Index
PRIMARY KEY	_id (ObjectId)

CRUD in MongoDB (Mongoose)

```
// CREATE
await User.create({ name: 'Suraj', email: 's@x.com' });
await new User({ name: 'Suraj' }).save();

// READ
await User.find({});           // all
await User.findById(id);       // by _id
await User.findOne({ email: 's@x.com' }); // first match
await User.find({ age: { $gt: 18 } }); // conditions

// UPDATE
await User.findByIdAndUpdate(id, { name: 'New' }, { new: true });
await User.updateMany({ role: 'free' }, { $set: { tier: 0 } });

// DELETE
await User.findByIdAndDelete(id);
await User.deleteMany({ inactive: true });
```

Query Operators — Most Important

- **\$gt / \$lt / \$gte / \$lte** — Greater than / less than
- **\$eq / \$ne** — Equal / not equal

- **\$in / \$nin** — Value in array / not in array
- **\$and / \$or / \$not** — Logical operators
- **\$exists** — Field exists or not
- **\$regex** — Pattern matching
- **\$set** — Update: set field value
- **\$push / \$pull** — Update: add/remove from array
- **\$inc** — Update: increment a number

Aggregation Pipeline

The aggregation pipeline processes documents through stages, like Unix pipes.

```
// Find average order value per user
db.orders.aggregate([
  { $match: { status: 'completed' } },          // filter
  { $group: { _id: '$userId',
              avgValue: { $avg: '$amount' } } }, // group + calc
  { $sort: { avgValue: -1 } },                  // sort desc
  { $limit: 10 },                              // top 10
  { $lookup: {                                  // JOIN users
      from: 'users', localField: '_id',
      foreignField: '_id', as: 'user'
    } },
  { $project: { avgValue: 1, 'user.name': 1 } } // shape output
]);
```

■ **Interview Tip:** "When would you use MongoDB over MySQL?" → When your data is document-like (different structures per record), you need horizontal scaling, or you're doing rapid prototyping. MySQL is better for strict relationships and complex joins.

ACID, Joins, Indexes, Transactions

ACID Transactions

Your notes use a great analogy for ACID — a light switch: either fully ON or fully OFF, never half-way. InnoDB (MySQL's default storage engine) supports all four ACID properties.

A — Atomicity

All operations in a transaction succeed, or ALL are rolled back. No half-transactions.

C — Consistency

A transaction brings the database from one valid state to another. Constraints are never violated.

I — Isolation

Concurrent transactions don't see each other's intermediate states.

D — Durability

Once committed, data survives crashes. Written to disk, not just RAM.

```
-- Transaction example
START TRANSACTION;
  UPDATE accounts SET balance = balance - 500 WHERE id = 1;
  UPDATE accounts SET balance = balance + 500 WHERE id = 2;
  -- If EITHER fails, both are rolled back
COMMIT;
```

Joins — The Most Important SQL Concept

```
-- INNER JOIN: only matching rows from both tables
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

-- LEFT JOIN: all users, even those with no orders
SELECT u.name, COALESCE(SUM(o.total), 0) AS spent
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id;

-- Subquery alternative
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders WHERE total > 1000);
```

Indexes — Why Queries Are Fast or Slow

An index is a **separate data structure (B-Tree)** that lets MySQL find rows without scanning the **whole table**. Without an index on email, a query like `WHERE email='x@y.com'` scans every row. With an index, it jumps directly to the matching rows.

```
-- Create index
CREATE INDEX idx_email ON users(email);

-- Composite index (column order matters!)
CREATE INDEX idx_user_date ON orders(user_id, created_at);

-- See if index is being used
EXPLAIN SELECT * FROM users WHERE email = 'x@y.com';
-- Look for: type=ref or type=const (good), type=ALL (bad – full scan)
```

■ **Interview Tip:** *Indexes speed up reads but slow down writes (index must be updated on INSERT/UPDATE). Don't index every column — only columns you frequently filter, sort, or join on.*

In-memory store, Strategies, TTL, Sessions

What is Redis?

Redis is an **in-memory key-value store** used for caching, session storage, pub/sub messaging, job queues, and rate limiting. Your emergency notes describe its main use cases accurately.

- Cache API responses to avoid hitting the DB on every request
- Store JWT blacklist (revoked tokens) for instant lookup
- Session storage (shared across multiple server instances)
- Rate limiting (increment counter per IP per minute)
- Job queue (BullMQ uses Redis as its backing store)
- Pub/Sub for real-time messaging between services

Caching Strategies

Cache-Aside (Lazy Loading)

App checks cache first. If miss, load from DB and store in cache. Most common pattern.

```
async function getUser(id) {
  const cached = await redis.get(`user:${id}`);
  if (cached) return JSON.parse(cached);          // cache HIT

  const user = await db.query('SELECT * FROM users WHERE id=?', [id]);
  await redis.setex(`user:${id}`, 3600, JSON.stringify(user)); // cache 1hr
  return user;
}
```

Write-Through

Write to cache AND DB simultaneously. Keeps cache always fresh. More write overhead.

```
async function updateUser(id, data) {
  await db.update(id, data);
  await redis.setex(`user:${id}`, 3600, JSON.stringify(data));
}
```

Cache Invalidation

Your notes mention this: cache invalidation is the problem of **removing or updating cached data when the underlying DB changes**. The naive fix: when you update a user in DB, delete their cache entry. The next request rebuilds it.

■ **Interview Tip:** "What are two hard problems in CS?" → Cache invalidation and naming things. Interviewers find this funny AND it shows you understand why caching is non-trivial.

Background jobs, Workers, Producers

Why Job Queues?

Your notes capture this perfectly: some tasks are too slow to do in the request-response cycle. Examples: sending emails, processing images, calling expensive APIs (like Claude), generating PDFs, sending WhatsApp messages. The solution: **decouple the task from the HTTP request**.

```
// WITHOUT queue (bad – user waits 5 seconds)
app.post('/send-otp', async (req, res) => {
  await sendSMSviaSlowAPI(req.body.phone); // blocks for 5s
  res.json({ ok: true });
});

// WITH queue (good – responds in <10ms)
app.post('/send-otp', async (req, res) => {
  await smsQueue.add('send', { phone: req.body.phone });
  res.json({ queued: true }); // instant!
});
// Worker picks it up asynchronously
```

BullMQ Architecture

Queue

A named Redis list. Holds jobs waiting to be processed.

Producer

Your API server. Adds jobs to the queue via `queue.add()`

Worker

A separate process. Pulls jobs from the queue and processes them.

Job

A unit of work with a name, data payload, and options (delay, retry, priority).

Full BullMQ Example

```
const { Queue, Worker } = require('bullmq');
const redis = { host: 'localhost', port: 6379 };

// Producer (in your Express route)
const emailQueue = new Queue('email', { connection: redis });

app.post('/register', async (req, res) => {
  const user = await createUser(req.body);
```

```

    await emailQueue.add('welcome-email', {
      to: user.email,
      name: user.name
    }, {
      attempts: 3, // retry 3x on failure
      backoff: { type: 'exponential', delay: 2000 }
    });
    res.status(201).json(user);
  });

// Worker (separate process: node worker.js)
const worker = new Worker('email', async (job) => {
  if (job.name === 'welcome-email') {
    await sendEmail(job.data.to, `Welcome ${job.data.name}!`);
  }
}, { connection: redis });

worker.on('completed', (job) => console.log(`Done: ${job.id}`));
worker.on('failed', (job, err) => console.error(`Failed: ${err}`));

```

■ **Interview Tip:** This is directly relevant to Dawa Saathi and Piilooopu. If an interviewer asks about your projects, explain how BullMQ decouples the WhatsApp API call from the HTTP response — that's a real architectural decision that shows senior thinking.

Deployment, Load Balancing, SSL

What is Nginx?

Nginx is a **high-performance web server and reverse proxy**. Your notes correctly describe it as an async, event-driven architecture (similar to Node.js) — it uses an event loop instead of spawning a thread per connection. In production, Nginx sits in **front of your Node.js app** and handles:

- Serving static files (HTML, CSS, images) directly — much faster than Node
- SSL termination (HTTPS) — handles TLS so your app doesn't have to
- Reverse proxying — forwards requests to Node on an internal port
- Load balancing — distributes traffic across multiple Node instances
- Caching — stores responses for repeated requests
- Rate limiting — blocks too many requests from one IP

Nginx Config — Production Setup

```
# /etc/nginx/sites-available/myapp

server {
    listen 80;
    server_name myapp.com www.myapp.com;
    return 301 https://$host$request_uri;    # redirect HTTP → HTTPS
}

server {
    listen 443 ssl;
    server_name myapp.com;

    ssl_certificate      /etc/letsencrypt/live/myapp.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/myapp.com/privkey.pem;

    # Serve static files directly
    location /static/ {
        root /var/www/myapp;
        expires 30d;
    }

    # Proxy everything else to Node.js
    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Nginx vs Apache vs Node as Server

	Nginx	Apache	Node (direct)
Concurrency model	Async event loop	Thread per request	Async event loop
Static files	Very fast	Fast	Slow
Memory (1000 users)	Low (~15MB)	High (~250MB)	Depends on app
Config style	Block-based	Directive-based	Code
Use in production	■ Standard	Legacy	■ Don't expose direct

EC2, IAM, S3, VPC, RDS

AWS Account Structure

Your notes show you understand: Root account has full power. Never use it day-to-day. Create IAM users with only the permissions they need (Principle of Least Privilege).

Core Services for a Backend Developer

EC2 (Elastic Compute Cloud)

Virtual machines. You run your Node.js app here. Choose instance type based on CPU/RAM needs. Use Ubuntu 22.04 LTS, install Node, clone your repo, use PM2 to keep it running.

IAM (Identity & Access Management)

Users, roles, policies. IAM roles are attached to EC2 instances to give them AWS permissions (e.g., so your Node app can write to S3 without hardcoding credentials).

S3 (Simple Storage Service)

Object storage for files, images, backups. Infinitely scalable. Used in Piiloopu for storing generated invitation images via Cloudinary — similar concept.

RDS (Relational Database Service)

Managed MySQL/PostgreSQL. Handles backups, failover, patching. More expensive than self-hosting but saves ops work.

VPC (Virtual Private Cloud)

Private network. Put your DB in a private subnet so it's not exposed to the internet. Only your EC2 can reach it. This is basic security hygiene.

Route 53

DNS service. Points your domain to your EC2 elastic IP.

CloudFront

CDN. Caches and serves your static assets (JS, CSS, images) from edge locations near users.

ElastiCache

Managed Redis/Memcached. Drop-in replacement for your self-hosted Redis.

Typical Production Stack (What Interviewers Expect You to Know)

```
Internet
  ↓
Route 53 (DNS)
  ↓
CloudFront (CDN for static, optional)
  ↓
EC2 (Nginx + Node.js app + PM2)
  ↓ ↓ ↓
RDS      ElastiCache    S3
(MySQL)  (Redis)        (Files)
```

■ **Interview Tip:** When asked about deployment, name this stack. Even if you've only deployed on a single EC2, understanding this architecture shows you can think in production terms.

How the Internet Works — DNS

Your notes describe DNS as 'the phone book of the internet'. Great analogy. When you type google.com:

- Browser checks its cache — has it resolved this recently?
- OS checks /etc/hosts and its own DNS cache
- Query goes to Recursive Resolver (your ISP or 8.8.8.8)
- Resolver queries Root nameserver → TLD nameserver (.com) → Authoritative nameserver (google.com)
- Returns IP address (e.g. 142.250.195.14)
- Browser connects to IP on port 443 (HTTPS) or 80 (HTTP)

■ **Interview Tip:** HTTPS uses port 443. HTTP uses port 80. DNS uses port 53. These are worth memorising.

Scalability Strategies

Vertical Scaling (Scale Up)

Bigger machine — more CPU, more RAM. Simple but has limits and is expensive.

Horizontal Scaling (Scale Out)

More machines behind a load balancer. How real web-scale works.

Load Balancing

Nginx or AWS ALB distributes incoming requests across multiple server instances.

Database Read Replicas

Write to primary DB, read from replicas. Handles read-heavy traffic.

Caching Layer

Redis in front of DB. Serves hot data from RAM instead of disk.

CDN

Edge nodes serve static assets from locations close to users. Reduces latency globally.

Message Queues

Decouple services. BullMQ/SQS/Kafka absorb traffic spikes.

Database Sharding

Partition data across multiple DB servers by user ID range. Extreme scale.

CAP Theorem (Distributed Systems)

In a distributed system, you can only guarantee 2 of 3: **Consistency** (all nodes see same data), **Availability** (every request gets a response), **Partition tolerance** (works despite network failures). Since network partitions are unavoidable, you choose between CP or AP.

- MongoDB: CP (consistent after partition, may be unavailable briefly)
- Cassandra: AP (always available, eventually consistent)
- Redis Cluster: CP

App Router, SSR, SSG, API Routes

Rendering Strategies

Your Next.js notes are the most typed/readable. Here's the core rendering model:

SSR — Server-Side Rendering

HTML is generated on the server on EVERY request. Data is always fresh.

Use when: Dynamic pages: dashboards, user profiles, real-time data.

```
async function Page() { const data = await fetch(url, {cache:'no-store'}); }
```

SSG — Static Site Generation

HTML is generated at BUILD TIME. Super fast — served from CDN.

Use when: Blogs, documentation, marketing pages that rarely change.

```
async function Page() { const data = await fetch(url); /* cached by default */ }
```

ISR — Incremental Static Regeneration

Static pages that revalidate in the background after a time interval.

Use when: Product pages, listings that change occasionally.

```
async function Page() { const data = await fetch(url, {next:{revalidate:3600}}); }
```

CSR — Client-Side Rendering

Data fetched in the browser after the page loads (useEffect + fetch).

Use when: User-specific data that doesn't need SEO.

```
// 'use client' + useEffect(() => { fetchData() }, [])
```

App Router vs Pages Router

- App Router (Next 13+): files in /app, uses Server Components by default
- Pages Router: files in /pages, all components are client-side by default
- Server Components: run on server, can be async, access DB directly, can't use hooks
- Client Components: 'use client' at top, can use hooks, run in browser
- layout.tsx: persistent UI that wraps child pages (nav, sidebar)
- loading.tsx: Suspense boundary — shown while page data loads
- error.tsx: error boundary for that route segment

API Routes (Route Handlers)

```
// app/api/users/route.ts
import { NextRequest, NextResponse } from 'next/server';

export async function GET(req: NextRequest) {
  const users = await db.query('SELECT * FROM users');
  return NextResponse.json(users);
}

export async function POST(req: NextRequest) {
  const body = await req.json();
  const user = await db.create(body);
  return NextResponse.json(user, { status: 201 });
}

// app/api/users/[id]/route.ts
export async function GET(req: NextRequest,
  { params }: { params: { id: string } }) {
  const user = await db.findById(params.id);
  return NextResponse.json(user);
}
```

■ **Interview Tip:** Common Q: "When should you use Route Handlers vs Server Actions?" → Route Handlers for REST APIs consumed by external clients. Server Actions for form mutations within the Next.js app — they avoid needing a separate API endpoint.

End of Guide — Good luck, Suraj. You've done the hard work. Trust your notes, trust your projects.